Security Recommendations

Security Recommendations

1. Human-verification / bot protection

- **Firebase App Check** (the most important one for this stack) — uses **reCAPTCHA Enterprise v3** under the hood. Once enforced, your `firebase-applet-config.json` apiKey becomes nearly useless to attackers, because requests without a valid attestation token are rejected *before* hitting Firestore. This alone neutralizes most of the "open database" damage even while rules are being fixed. Note: your config already has a `recaptchaSiteKey` field (currently empty) — the plumbing was anticipated but never wired up.
- **reCAPTCHA v3 on the login form** — invisible score-based challenge, only escalating to a visible challenge on low scores. Avoid v2 checkbox on a kiosk (children/staff get stuck on image puzzles).
- **Firestore rules + App Check enforcement combined** — rules can require `request.auth != null && firestore.get(/databases/$(database)/documents/__appcheck__...` patterns, or simpler: enable App Check *enforcement* on Firestore in the console, so even valid-rule-but-bot traffic is blocked.

2. 2FA / MFA

- **Firebase Authentication Identity Platform MFA** — supports **TOTP** (Google Authenticator, etc.) as a second factor for staff/admin. This is the right choice here over SMS-based 2FA (SMS is SIM-swap vulnerable and costs money per message). Make it **mandatory for `admin` role**, optional-to-required for staff.
- **Re-authentication (step-up auth) for sensitive actions** — before approving a child release, editing staff accounts, or bulk-exporting data, require the user to re-enter their password (Firebase supports `reauthenticateWithCredential`) or a **time-rotating TOTP code**. This is the proper fix for the staff-approval modal — better than a static PIN.
- **2FA on the Google/Firebase Console accounts themselves** — all console users (the real crown jewels: rules, Auth config, database) should have 2FA + hardware security keys. Least-privilege IAM: most staff should have **no** console access at all.

3. Identity & account hardening

- **Password policy** — Firebase Auth lets you enforce minimum length/complexity server-side. Right now nothing stops `password`.
- **Email enumeration protection** — enable it in Firebase Auth settings (returns generic errors).
- **Restrict Google sign-in** to an allowlisted domain (or remove the popup entirely — it's currently an escalation path, per H2 in the report).
- **Custom claims for roles** — move `role` out of Firestore docs into JWT custom claims set by an Admin SDK Cloud Function. Clients can't tamper with them, and Firestore rules can enforce them

(`request.auth.token.role == 'admin'`).
- **Session revocation** — ability for admins to force-logout a user (`revokeRefreshTokens`), e.g., when a staff member leaves.

4. Session & kiosk-specific controls (this product's blind spot)

- **Idle auto-lock** — kiosk should lock after 30–60s of inactivity; dashboards after 5–10 min. Currently sessions persist forever.
- **Kiosk mode browser** — run the kiosk in a locked-down browser (Chrome kiosk mode / dedicated profiles): no DevTools, no extensions, no address bar. This matters because the entire app is client-side and localStorage contains the roster + credentials.
- **Per-device identity** — the kiosk should not use a personal staff account; give it a dedicated, least-privilege "kiosk" service identity so a compromised kiosk ≠ compromised admin.
- **Physical verification at check-out** — this is arguably the biggest *new* control to add: **photo capture of the pickup person** and/or **signature capture** stored with the release record, and/or **QR-code student cards** scanned instead of typed names (kills impersonation + the name-autocomplete privacy leak).
- **Privacy screen / auto-blur** — mask roster and phone numbers when the kiosk is idle.